

RELATÓRIO DE DESENVOLVIMENTO: Web Scraping - Versão Básica

ALUNO: Willian Nunes dos Santos

DISCIPLINA: Business Intelligence

1. INTRODUÇÃO E OBJETIVO

O objetivo deste projeto foi desenvolver uma aplicação básica em Python para extrair automaticamente (Web Scraping) dados de preços e títulos de livros de um website de e-commerce simulado. A aplicação visa automatizar a coleta de dados de uma única página. **Website Alvo:** Books to Scrape. **Justificativa:** O site foi escolhido por ser um ambiente seguro para testes de scraping, permitindo a extração de dados sem violar termos de serviço e possuindo uma estrutura HTML de fácil compreensão.

2. ANÁLISE DA ESTRUTURA HTML

Para realizar a extração, foi necessário inspecionar o código-fonte da página. Identificou-se os seguintes padrões:

- **Container do Produto:** Cada livro está encapsulado em uma tag <article> com a classe product_pod.
- **Título:** Localizado dentro de um <h3>, dentro de uma tag <a> no atributo title.
- **Preço:** Localizado em uma tag <p> com a classe price_color.

3. FERRAMENTAS E BIBLIOTECAS UTILIZADAS

- **Python 3:** Linguagem base.
- **Requests:** Para realizar as requisições HTTP (baixar o HTML da página).
- **BeautifulSoup (bs4):** Para fazer o "parsing" (análise e navegação) da árvore HTML.
- **Pandas:** Para estruturar os dados em tabela e exportar para CSV.

4. ALGORITMO DE EXTRAÇÃO

O algoritmo foi implementado seguindo a lógica: A) Definição da URL alvo. B) Requisição GET para obter o conteúdo HTML e conversão para objeto

BeautifulSoup. C) Busca de todos os elementos <article> com a classe product_pod. D) Iteração sobre cada livro encontrado para extrair o título e o texto do preço. E) Armazenamento dos dados em uma lista de dicionários. F) Conversão para DataFrame e exportação final para o arquivo CSV.

5. RESULTADOS E DESAFIOS

O script foi executado com sucesso capturando os dados da página principal.

- **Formato de saída:** Arquivo relatorio_livros_toscrape.csv contendo as colunas 'Nome do Livro' e 'Preço'.
- **Desafios enfrentados:** Garantir que o arquivo CSV lidasse corretamente com caracteres especiais, sendo resolvido através do uso do parâmetro encoding='utf-8-sig' na exportação pelo Pandas.

6. CÓDIGO FONTE

Python

Importar as bibliotecas necessárias

import pandas as pd

import requests

from bs4 import BeautifulSoup

#-----

Corpo principal da aplicação

#-----

Endereço onde iremos buscar os dados a serem 'raspados'

url = 'http://books.toscrape.com/'

Executar a chamada ao endereço carregado na variavel url, carregando o resultado para a variavel pagina

pagina = requests.get(url)

```
# Traduzir o conteudo de 'pagina' para o formato html
```

```
soup = BeautifulSoup(pagina.content, 'html.parser')
```

```
# Filtro do html separando efetivamente os blocos de produtos
```

```
lista_produtos_html = soup.find_all('article', class_='product_pod')
```

```
# Criar uma lista vazia para guardar nossos dados
```

```
dados_extraidos = []
```

```
# Varrer cada item encontrado para tirar Título e Preço
```

```
for produto in lista_produtos_html:
```

```
    # Pegar o titulo
```

```
    titulo = produto.find('h3').find('a')['title']
```

```
    # Pegar o preço
```

```
    preco = produto.find('p', class_='price_color').text
```

```
# Adicionar na nossa lista
```

```
dados_extraidos.append({
```

```
    'Nome do Livro': titulo,
```

```
    'Preço': preco
```

```
})
```

```
#-----
```

```
# Tecnologia: Transformar a lista de dados para um dataframe, usando a biblioteca  
de pandas
```

```
dataframe = pd.DataFrame(dados_extraidos)
```

```
#Salva a tabela do dataframe em uma planilha excel
```

```
dataframe.to_csv('relatorio_livros_toscrape.csv', index=False, encoding='utf-8-sig')
```

7. OBSERVAÇÕES

Este é um script estático e de nível introdutório, ideal para a compreensão da biblioteca BeautifulSoup e do mapeamento de tags HTML.